pg-flight-recorder

A Black Box for PostgreSQL

David A. Ventimiglia

Supabase

Lightning Talk | 2024

The Problem

> *"What was happening in my database?"*

THE INCIDENT

3:00 AM

Users complaining, dashboards flashing red.

You Wake Up

Check PostgreSQL... everything looks fine now.

The evidence vanished.

The incident ended, leaving no trace.

POSTGRESQL REALITY

PostgreSQL forgets the past

- `pg_stat_activity`
  Shows **now**, not **then**.
- No built-in history of wait events, locks, or active queries.

The Solution

Flight Recorder Concept

"Always recording, ready when you need it"

Like an airplane's black box - you don't think about it until you need it

Server-side

Runs inside PostgreSQL completely autonomously via `pg_cron`.

NATIVE

Zero-config

No complex setup or external agents required. Just run one command.

```
psql -f install.sql
```

Production-safe

Battle-tested at Supabase scale with minimal overhead.

RELIABLE

What It Captures

CATEGORY	METRICS CAPTURED
Wait Events	What backends are waiting on (IO, locks, LWLocks, etc.)
Active Queries	Running queries with session age, state, and transaction info
Locks	Lock contention and blocking chains identification
WAL & Checkpoints	Write-ahead log generation rates and checkpoint activity stats
I/O Timing	Storage latency breakdowns by backend type (read/write)
Top Queries	Slowest and most frequent queries (via pg_stat_statements)
Table/Index Stats	Access hotspots, dead tuple bloat, and unused indexes
Config Changes	Parameter drift detection and restart requirement tracking

How It Works

pg_cron

Central scheduler running inside PostgreSQL background worker.

Every 3m

Every 5m

UNLOGGED

Ring Buffers

Captures **Sampled Activity** (wait events, sessions) in memory-mapped tables.

Aggregation

7-DAY RETENTION

Archives

Aggregates sampled data + stores raw archives for historical analysis.

30-DAY RETENTION

Snapshots

Full system state capture: WAL generation, checkpoint stats, I/O timing, and top queries.

Fast Path (High Frequency)

System State (Medium Frequency)

Production Safe

PERFORMANCE IMPACT

Measured Overhead

~0.02%

CPU Usage

Median collection	23 ms
P95 collection	31 ms
Collections / day	480

BUILT-IN PROTECTION

Load Shedding

automatically pauses collection when connection utilization exceeds `70%`.

Circuit Breaker

Aborts and skips subsequent runs if individual collections become too slow.

Storage Guardrails

Auto-disable mechanism triggers if storage usage hits `10 GB` limit.

Non-Blocking

Guaranteed `0%` DDL blocking. Tested against 202 sensitive operations.

Quick Demo: The Report

RUN ONE COMMAND

psql console

```
SELECT
flight_recorder.report('1
hour');

_ Waiting for input...
```

ANALYST TIP

Paste the output into **ChatGPT** for instant root cause analysis!

REPORT OUTPUT INCLUDES

Anomalies Detected
Checkpoints, buffer pressure, spikes

Wait Event Summary
Aggregated wait times by type

Lock Contention
Blocking chains & deadlocks

Long-running Tx
Idle in transaction & old xids

Config Changes
Parameter drift detection

Table Hotspots
Sequential scans & index efficiency

```
Report ID:    rpt_20231024_001
Duration:     1 hour (02:00 - 03:00)

WARNING: High Checkpoint Frequency Detected
Wait Events Distribution:
  IO:DataFileRead ......... [||||||||||] 45%
  LWLock:BufferContent .... [|||||  ] 25%
```

Real Investigation: Compare

QUERY

```
SELECT * FROM flight_recorder.compare(
'2024-01-15 02:00', -- before incident (baseline)
'2024-01-15 03:00' -- during incident (spike)
);
```

RESULT SET

METRIC	BEFORE	DURING	DELTA
checkpoint_occurred	false	true	-
bgw_buffers_backend_delta	0	12,847	**+12,847**
temp_bytes_delta	0	2.1 GB	**+2.1 GB**

DIAGNOSIS

Heavy checkpoint occurred during peak load + client backends forced to write dirty pages directly to disk.

Suggestion: Increase max_wal_size

Suggestion: Tune bgwriter

Anomaly Detection

ANOMALY TYPE	WHAT IT MEANS
CHECKPOINT_DURING_WINDOW	I/O spike caused by heavy checkpoint activity during peak hours
BUFFER_PRESSURE	Backends bypassing bgwriter, forcing direct disk writes
LOCK_CONTENTION	High number of sessions blocked waiting on locks
XID_WRAPAROUND_RISK	Transaction ID consumption approaching safety limits
IDLE_IN_TRANSACTION	Sessions holding open transactions, blocking vacuum cleanup
DEAD_TUPLE_ACCUMULATION	Table bloat building up due to insufficient vacuuming
REPLICATION_LAG_GROWING	Replica falling behind primary, risking data freshness

TOTAL COVERAGE

12 distinct issue types auto-detected out of the box

- Configurable thresholds
- JSON output

Installation

GET STARTED

bash

INSTALL

```
$ psql -f install.sql
# Collection starts automatically
```

Includes automatic schema setup and job scheduling.

UNINSTALL

bash

REMOVE

```
$ psql -f uninstall.sql
```

REQUIREMENTS

PostgreSQL
Versions 15, 16, or 17

pg_cron
Extension for scheduling
REQUIRED

pg_stat_statements
Query analysis metrics
OPTIONAL

The recorder runs entirely within the database. No external agents or sidecars required.

Try It Today

GITHUB REPOSITORY
`github.com/supabase/pg-flight-recorder`

bash — root@db-prod:~

```
$ git clone https://github.com/supabase/pg-flight-recorder
$ psql -f install.sql
# ... wait for an incident ...
$ psql -c "SELECT flight_recorder.report('1 hour');"
```

MIT Licensed · Open Source · Production-Proven

Questions?

David A. Ventimiglia | Supabase